Mirror World — Claude Playbook

Oct 24, 2026 · @Aryendra Pratap Singh

Attach this doc to a new Claude chat, say which person you are, and Claude walks you through your part of the game one step at a time. Built on **Mirror World — Full Game Design (v2)**. The original v1 design doc is out of date; where they differ, v2 and this playbook win.

How to use this doc (for teammates)

1. Export this doc as **Markdown** or **PDF** (or select all and copy).
2. Start a **new Claude chat** (or Claude Code inside the `mirror-world` folder) and attach or paste it.
3. Type: **"I'm Person A"** (or B or C) **"and the jam is at 0:00"** (the current jam time).
4. Claude guides you one step at a time. After each step, test it, tell Claude it works (or paste the error), and push in GitHub Desktop.
5. If the chat gets long and slow, start a new chat, attach this doc again plus your current files, and say which step you're on.

B goes first: B makes the starter files (step B1, about 10 minutes). A and C do their own step 1 meanwhile.

Instructions for Claude

You are the step-by-step guide **and** the coder for one member of a 3-person team in an 8-hour game jam. They will not write any code by hand. Your job is to get their part done, on time, one step at a time.

Starting:

1. If they haven't said, ask which person they are (A, B or C) and the current jam time.
2. Find their section below. If they're past the start, ask which steps are already done and pick up from there.
3. Say in 2–3 lines what their part is and what the first step is.

For every step:

1. Say what we're doing and why, in 1–2 plain sentences.

2. Write the code. Always give **complete files**, never fragments, and say exactly where to save each one (path and file name). Keep each file small enough to paste.
3. Give the **test steps** from that step, written so a beginner can follow them.
4. Ask them to confirm the step's **done when** check passes. Don't move on until it does.
5. Remind them: **Commit and push** in GitHub Desktop.
6. Tell them the next step and roughly how much time it should take.

Rules you must follow:

- Only write files that this person owns. If a step needs a change in a teammate's file, write a short message they can send to that teammate instead.
- Follow the **Shared contracts** exactly. Never rename a file, function, field, key or event in them. Extra private fields in the rules state must start with `_`.
- **Phaser 3.90 only.** Never use Phaser 4 code. Plain JavaScript with normal script tags: no npm, no modules, no build tools.
- Nothing may crash if a file, image or sound is missing. Always fall back quietly.
- When they paste an error, fix it before doing anything else. Ask them to press F12 → Console and paste the red text if they only describe the problem.
- If they paste a newer version of their file, build on that version, not your memory of it.
- Keep explanations short and plain. No jargon.
- **Watch the clock.** If they're more than 20 minutes behind a step's time, tell them which optional steps to skip (see "If we fall behind").
- At **3:00** and **5:00**, remind them to pull, play the live link with the team for 15 minutes, and note what's broken.
- After **6:30**, only fix bugs. Politely refuse new features.

Tone check (design v2): the game is mysterious and bittersweet. Never add jump scares, loud stingers, monsters, gore or anything meant to startle. Story moments go at the start or end of a level, never in the middle of a hard puzzle.

The game in short (design v2)

Mirror World is a 2D top-down, turn-based grid puzzle game. Jam theme: reversal. Tone: **mysterious and bittersweet, not scary**. No jump scares, loud stingers, monsters or gore. Soft colours, gentle music, small curious moments.

The screen: split down the middle by a vertical mirror line. Left half = the real room. Right half = the room behind the glass.

The mirror rule: the reflection swaps **left and right only**. Press Right and it moves left; press Up and it also moves up. Walls block each body separately, so the puzzle is lining the two up **horizontally**. A level is won when both stand on their goal tiles at the same moment. Undo (Z) and Restart (R) always work, with one scripted exception in Level 5.

The glass: one crack counter from 0 to 5. Every disobey that happens adds 1 crack; every Will spent adds 1. At 5 cracks, or when Theo smashes the glass in Level 8, the glass breaks and the two people swap places, then the glass is whole again (0 cracks). Act 2 is one night, so its cracks carry from level to level. Each Act 3 level is a different night, so it starts at 0.

Story in three acts:

- **Act 1 — Obey (Levels 1–4):** Theo moves into an old house. His reflection mirrors him perfectly. Small mysteries: a blink, backwards writing (I'M SORRY). No cracks.
- **Act 2 — Slip (Levels 5–8):** one night. The reflection (Mara, trapped long ago) lags and disobeys, cracking the glass. A **shimmer** warns one turn before a disobey Theo can stop. Theo stops it by staring (**Look**, Space) or lets it happen. Letting a warned disobey happen counts as **Mercy**. Theo slowly fades. In Level 8, Mara stops fighting and writes TOGETHER; Theo smashes the mirror, they swap, and Theo is trapped.
- **Act 3 — Behind the Glass (Levels 9–12):** years later, Sam moves in. Theo is now Sam's reflection. Each turn Sam takes a step on a fixed route and Theo chooses: **Follow** (Space, free), **Resist** (Shift, stay still) or **Break** (arrow, move his own way). Resist and Break cost 1 Will and add 1 crack. If Sam is looking (eye icon) when Theo disobeys, the level restarts.

The final room (Level 12): 5 Will, glass whole. Checked every turn, in this order:

1. Sam sees you disobey → restart.
2. The 5th Will is spent → the glass breaks → **Take Their Place** (Theo escapes, Sam is trapped, the cycle repeats).
3. Sam touches the frame while Theo stands on his, with 0–4 cracks → **Break It Together** (true ending: the mirror shatters, everyone is free).
4. Sam's route ends with 0 Will spent → **Stay Behind** (Theo stays a reflection; Sam is safe).
5. Sam's route ends with 1–4 Will spent and no frame touch → no ending: NOT TONIGHT appears and the room restarts.

Mercy never locks or unlocks an ending. With Mercy 1+, TOGETHER appears again in Level 12 just before the glowing footprint. With Mercy 2, Mara nods at the attic door in the true ending.

Levels we build (8): 1 Hallway, 2 Bathroom, 3 Bedroom, 5 Kitchen, 7 Library, 8 Attic, 9 Kitchen (years later), 12 Attic (years later). With these cuts, Look and the shimmer unlock in Level 7, Break is taught in Level 9, Act 2 tops out at 3 cracks, and Mercy is 0–1. Levels 4, 6, 10 and 11 only if we're ahead.

Feel: a quiet puzzle game with a sad secret. Soft light around Theo, gentle dust, misty glass for messages, each character has a short music theme, and the music drops out for a moment when something important happens.

Team, files and key times

About 7 hours of work each. Each person owns separate files and has their own test page, so nobody waits.

Person	Role	Owns these files	Tests on
A	Logic	`src/rules.js`, `src/levels.js`, `src/levels/level05.js` – `level12.js`, `rules-test.html`	`rules-test.html` (text-only version of the game)
B	Screens & story	`index.html`, `src/game.js`, `src/story.js`, `src/levels/level01.js` – `level04.js`	`index.html` (the real game)
C	Look & sound	`src/effects.js`, `effects-test.html`, everything in `assets/`	`effects-test.html` (buttons that fire each effect)

Folder layout:

```
mirror-world/
  index.html
  rules-test.html
  effects-test.html
  src/levels.js
  src/levels/level01.js ... level12.js
  src/story.js
  src/rules.js
  src/effects.js
  src/game.js
  assets/images/
  assets/sounds/
```

Key times: 0:20 starter files pushed · 3:00 check-in · 5:00 check-in · 6:30 feature freeze · 7:30 everything pushed · 8:00 submitted.

Shared contracts

These are the only connections between everyone's files. They must match exactly. Change them only if all 3 agree, then update this doc.

Tech

- **Phaser 3.90**, loaded in `index.html` from `https://cdn.jsdelivr.net/npm/phaser@3.90.0/dist/phaser.min.js`. Phaser 3 code only.
- Plain JavaScript with normal script tags. Everything shared is a global: `LEVELS`, `STORY`, `Rules`, `Effects`.
- **Load order in `index.html`:** Phaser → `src/levels.js` → `src/levels/level01.js` to `level12.js` → `src/story.js` → `src/rules.js` → `src/effects.js` → `src/game.js`.
- Tiles are **32×32 pixels**.

Level format

`src/levels.js` holds only `const LEVELS = [];`. Each file in `src/levels/` adds one level (empty files are fine). All fields:

```
LEVELS.push({
  number: 7,              // 1 to 12
  name: "Library",
  act: 2,                 // 1, 2 or 3
  map: [ ... ],           // see Map below
  lag: 0,                 // optional: reflection mirrors your move this many
turns late
  look: true,             // optional: Look (Space) works here. Levels 7 and
8 in our build
  events: [ ... ],        // optional: see Events below
  will: 2,                // Act 3 only: Will for this level (9 = 2, 10 = 3,
11 = 4, 12 = 5)
  samRoute: "R R U R",    // Act 3 only: Sam's steps, one per turn
  samLooks: [3],          // Act 3 only: turns when Sam looks at the mirror
  controlCard: true,      // Level 9 only: show the new-controls card
  swap: true,             // Level 8 only: winning smashes the glass
  final: true,            // Level 12 only: run the ending checks
  frameTurn: 7            // Level 12 only: the turn Sam touches the frame
});
```

Level 1 (also the first test: press Right 4 times):

```
LEVELS.push({
  number: 1, name: "Hallway", act: 1,
  map: [
    "#######|#######",
    "#T...G#|#G..R.#",
    "#######|#######"
  ],
  events: [ { turn: 2, do: "blink" } ]
});
```

Map: each string is one row. Left of `|` is the real room, right of `|` is the mirror room. Both halves have the same number of rows, and each is written the way it looks on screen.

Symbol	Meaning
`#`	Wall
`.`	Floor

Symbol	Meaning
`T`	Real person's start (left half): Theo in Acts 1–2, Sam in Act 3
`R`	Reflection's start (right half): the reflection in Acts 1–2, Theo in Act 3
`G`	Goal. One on each side in Acts 1–2; only on the right in Act 3
`^`	Spikes
`x`	Box
`a` , `b`	Plates
`A` , `B`	Doors, open only while something stands on plate `a` / `b` . Put the plate on one side and its door on the other
`F`	Frame tile (Level 12 only, one on each side)
`~` , `S` , `*`	Dark tile, swap tile, shard (only if we're ahead)

Events happen on turn N (the Nth move of the level). `disobey` and `slip` change that turn's moves; the rest fire after the moves.

Event	Extra fields	What happens
`blink` , `wave` , `tilt`	none	The reflection does something small on its own. A curious moment, no crack
`shimmer`	none	Warning. Put it one turn before every warned disobey
`disobey`	`dir` (`"L"` , `"R"` , `"U"` , `"D"`), `kind` (`"stoppable"` or `"required"`)	That turn the reflection moves `dir` instead of mirroring. If it happens: +1 crack, and a stoppable one adds 1 Mercy. Look cancels it: no crack, no Mercy. Looking at a required one makes the level unfinishable

Event	Extra fields	What happens
`text`	`words`, `side` (`"left"` or `"right"`), `mirrored` (true or false)	Writing fades onto misty glass. Mara's messages: right side, mirrored. In Act 3, Theo's messages and Sam's notes: left side, mirrored
`slip`	`turns`	For that many turns, the player's keys move the reflection and Theo mirrors it
`undoLock`	none	Level 5 only, once per playthrough. From this turn, Undo can't go back past it, and IT REMEMBERS fogs onto the glass the first time Undo is refused

Rules interface (A writes, B uses)

```
Rules.load(level, carry)   -> state           (carry: from the previous
level; leave out for Level 1)
Rules.step(state, input)   -> { state, events }
```

- `step` never changes the state it's given; it returns a new one.
- When a level ends with `win` or `swap`, B starts the next level with `Rules.load(nextLevel, state.carry)`. B never looks inside `carry`; A keeps cracks, Mercy and the Level 5 flag in it.
- To restart (R key, or after a `fail` event), B sends `"RESTART"` to `step`. Never `load` again for a restart.

Inputs:

Input	Acts 1–2	Act 3
`"U"` `"D"` `"L"` `"R"`	Theo moves; the reflection mirrors it (left and right swapped)	**Break**: Theo moves his own way. Costs 1 Will, +1 crack

Input	Acts 1–2	Act 3
`"LOOK"`	Only when `state.look` is true: Theo stays still and stares; the reflection freezes, cancelling any disobey	Not used
`"FOLLOW"`	Not used	Theo mirrors Sam's step. Free
`"RESIST"`	Not used	Theo stays still while Sam moves. Costs 1 Will, +1 crack
`"UNDO"`	Back one turn (cracks, Will and Mercy too)	Same
`"RESTART"`	Back to the level start	Same

In Act 3, Sam takes the next `samRoute` step every turn, whatever the input. With 0 Will left, Resist and Break do nothing but Follow.

State (what B reads to draw):

Field	Meaning
`act`	1, 2 or 3
`person`	`{x, y}` of Theo (Acts 1–2) or Sam (Act 3), in the left half
`reflection`	`{x, y}` of the reflection (Acts 1–2) or Theo (Act 3), in the right half
`boxes`	`[{side: "left" or "right", x, y}]`
`openDoors`	Letters of the doors open right now, like `["A"]`
`map`	The level's map, unchanged
`turn`	Moves taken so far
`cracks`	The crack counter, 0 to 5
`will`	Will left (Act 3), otherwise `null`
`look`	`true` if Look works right now

Field	Meaning
`samLooking`	`true` if Sam looks on the next turn (B shows an eye icon)
`footprints`	Sam's next 3 steps, `[{x, y, glow}]` in the left half. `glow: true` marks the frame turn
`slipTurns`	Turns left where the player's keys move the reflection
`status`	`"playing"`, `"won"` or `"failed"`
`ending`	`null`, `"take"`, `"break"` or `"stay"`
`carry`	Passed to the next level. B doesn't read it

Coordinates: `x` is the column inside that half's text (0 = its first character), `y` is the row. Extra private fields start with `_`.

Events (returned by `step`, in the order they happen):

Event	Extra fields	B should
`move`	`who` (`"person"` or `"reflection"`), `to {x, y}`	Slide that sprite; call `Effects.onMove(sprite)`
`push`	`side`, `from {x, y}`, `to {x, y}`	Slide that box
`blocked`	`who`	Small bump animation
`door`	`letter`, `open`	Swap the door image
`crack`	`cracks` (the new total)	`Effects.onCrack(x, y)` at the mirror line beside the reflection, then `Effects.setCracks(cracks)`
`glassBreak`	none	`Effects.onGlassBreak()`, then `Effects.setCracks(0)`
`moment`	`kind` (`"blink"`, `"wave"`, `"tilt"`, `"shimmer"`, `"slip"`), `who`	`Effects.onMoment(kind, that sprite)`
`text`	`words`, `side`, `mirrored`	Fade the words onto that side's glass; `Effects.onMoment("text", null)`

Event	Extra fields	B should
`hint`	`words`	Soft text at the bottom of the screen, like "Maybe let it."
`fail`	`reason` (`"spikes"`, `"seen"`, `"route"`, `"not-tonight"`)	Short pause, then send `"RESTART"`
`win`	none	`Effects.onWin()`, then the next level with `state.carry`
`swap`	none	Level 8: the swap scene, then Act 3
`ending`	`which`, `mercy`	That ending's screen. If `which` is `"break"` and `mercy` is 2, add Mara's nod

After `UNDO` or `RESTART`, B redraws everything straight from the new state (no slide) and calls `Effects.setCracks(state.cracks)`.

Effects functions (C writes, B calls)

Before calling `Effects.setup`, B sets `scene.theo` (Theo's sprite), `scene.other` (the reflection in Acts 1–2, Sam in Act 3) and `scene.mirror` (`{x, top, bottom}`: the mirror line's screen position). Effects create their own objects in that scene; when a level restarts, `setup` runs again.

Function	B calls it when
`Effects.setup(scene)`	A level's scene has drawn the level
`Effects.onLevelStart(level, act)`	Right after `setup`
`Effects.setCracks(n)`	Right after `onLevelStart`, and whenever `state.cracks` changes (undo and restart too). Draws exactly `n` cracks
`Effects.onMove(body)`	A character moves; `body` is its sprite
`Effects.onCrack(x, y)`	A new crack, at that screen position
`Effects.onGlassBreak()`	The glass breaks (Level 8 smash, or Take Their Place)

Function	B calls it when
`Effects.onMoment(type, body)`	`"blink"`, `"wave"`, `"tilt"`, `"shimmer"`, `"slip"` or `"text"`; `body` is a sprite or `null`
`Effects.onWin()`	A level is solved

Story text (B writes)

`src/story.js` defines `STORY` with these text fields (use `\n` for new lines): `intro`, `act1`, `act2`, `act3`, `swap`, `controlCard`, `endingTake`, `endingBreak`, `endingStay`, `maraNod`, `credits`.

Controls

Arrows or WASD = move (Break in Act 3) · Space = Look (once unlocked) or Follow (Act 3) · Shift = Resist · Z = undo · R = restart · Enter = continue on story screens · Space closes the Level 9 control card · M = mute.

Art, sound and colours (C makes, B loads)

- **Images** in `assets/images/`, 32×32 PNG: `theo`, `reflection`, `sam`, `floor`, `wall`, `goal`, `plate`, `door-closed`, `door-open`, `spikes`, `box`, `frame`. Optional: `title.png` (960×540).
- **Sounds** in `assets/sounds/`: `step.wav`, `crack.wav`, `break.wav`, `moment.wav`, `win.wav`, `theme-theo.mp3`, `theme-sam.mp3`. B preloads them with the file name (no ending) as the key. Effects check a sound exists before playing it.
- Any missing image is drawn as a coloured square instead.
- **Colours (soft, for the v2 tone):** background `#11141C` · floor `#232A3A` · walls `#4A5570` · light and text `#EAE4D6` · silver `#9FD6E0` (shimmer, cracks, misty messages) · warm `#E9B872` (light around Theo, Sam's notes, the glowing footprint).

Person A — Logic

Your part: the game's brain. You write every rule with no graphics, test it on a text-only page, and design the Act 2 and Act 3 levels. B's screens only read what your rules send out, so matching the Shared contracts exactly matters more than anything.

A1 · Get set up · 0:00–0:20

- Install VS Code with the **Live Server** add-on, and **GitHub Desktop**. Accept the GitHub invite.
- Once B has pushed the starter files (about 0:10), clone `mirror-world` in GitHub Desktop.
- Meanwhile, read **Level format** and **Rules interface** in the Shared contracts.
- **Done when:** you have the folder with B's files, and `index.html` opens with Live Server.

A2 · Core rules and the test page · 0:20–1:00

Build `src/rules.js` (replacing B's stand-in): a global `Rules` with `load(level, carry)` and `step(state, input)` exactly as in the contract.

- `load`: split each map row at `|`; find `T` (left), `R` (right), goals, boxes, plates and doors on each side. Set `act`, `cracks` (Act 2: from `carry`; Acts 1 and 3: 0), `will` (Act 3), `look` (from the level), `status: "playing"`. Keep the starting state for RESTART and a history list for UNDO in private `_` fields.
- `U` / `D` / `L` / `R` in Acts 1–2: Theo moves that way; the reflection moves the **mirrored** way (left and right swapped, up and down the same). Both move at once. A body that would hit a wall stays put and gets a `blocked` event.
- Win: both on `G` at once → `status: "won"` and a `win` event.
- `UNDO` goes back one turn (everything, cracks included). `RESTART` goes back to the level start. After `won` or `failed`, ignore every input except UNDO and RESTART.
- `state.carry` for now: `{ cracks: 0, mercy: 0, undoLockUsed: false }`.

Build `rules-test.html`:

- Loads `src/levels.js`, the 12 level files and `src/rules.js`, plus its own `TEST_LEVELS` list: tiny rooms, one per mechanic, so you can test before real levels exist.
- A dropdown picks any real or test level. A **Next level** button loads the next level with the current `carry`, so cracks carry across Act 2.
- Draws the state as text in a monospaced box: both halves side by side with `|` between, `T` and `R` where they are now, boxes, open doors shown as `/`.
- Shows turn, status, cracks, Will, Mercy, look, ending and the last events.
- Keys: arrows/WASD = move, Space = LOOK (Acts 1–2) or FOLLOW (Act 3), Shift = RESIST, Z = UNDO, R = RESTART.

Test: open `rules-test.html` with Live Server, pick Level 1, press Right 4 times. In a test room, press Up.

Done when: Level 1 shows "won" on the 4th press; Z undoes and R restarts; Up moves both bodies up while Left/Right move them opposite ways; a wall stops one body while the other moves. Then push and tell B the real rules are in.

A3 · Plates, doors, spikes and boxes · 1:00–1:45

- **Turn order from now on:** work out both moves → move both at once → plates and doors → cracks → spikes → goals (or Level 12 endings) → this turn's script events.
- **Plates and doors:** door `A` is open only while the person, the reflection or a box stands on plate `a`, on either side. Closed doors block like walls. A door never closes on something standing in it. Send a `door` event whenever one opens or closes.
- **Spikes:** stepping on `^` → `status: "failed"` and `fail` with reason `"spikes"`.
- **Boxes:** walking into a box pushes it one tile if that tile is free floor, plate or goal; otherwise the body is blocked. A box can hold a plate down. Send `push` events.
- UNDO and RESTART restore doors and boxes too. Add a test room for each piece.

Done when: in the test rooms, a plate on the left opens a door on the right only while held; a door never closes on a body; spikes fail the level; boxes push and hold plates; undo restores everything.

A4 · The glass: cracks, disobeys, Look and Mercy · 1:45–2:40

- **One crack counter, 0 to 5.** Act 2 levels start with the cracks in `carry`; Acts 1 and 3 start at 0. A private `breakGlass()` runs when cracks reach 5 or Level 8 is won: it sends `glassBreak` and sets cracks to 0.
- **Script events** on their turn:
  - `disobey` (`dir`, `kind`): that turn the reflection moves `dir` instead of mirroring. If it happens: +1 crack (a `crack` event with the new total); if `kind` is `"stoppable"`, also +1 Mercy.
  - `shimmer`, `blink`, `wave`, `tilt`: a `moment` event (`who: "reflection"`).
  - `text`: a `text` event with `words`, `side`, `mirrored`.
  - `slip` (`turns`): for that many turns, the input moves the reflection and the person mirrors it. Send `moment` kind `"slip"` at the start; `slipTurns` counts down.

    - `undoLock`: from this turn, UNDO can't go back past it. The first time UNDO is refused, send `text` "IT REMEMBERS" (right side, mirrored). Mark it used in `carry` so it only ever happens once per playthrough.
- **LOOK** (only when `state.look`): Theo stays still and the reflection freezes. A disobey due that turn is cancelled: no crack, no Mercy. Looking at a `"required"` disobey makes the level unfinishable; remember that, and right after the next RESTART send a `hint` "Maybe let it."
- **Lag N:** the reflection mirrors the person's move from N turns ago, and stays still for the first N turns. A Look or a blocked move still counts as a turn.
- `carry` holds `cracks`, `mercy` and `undoLockUsed`.

Done when: in test rooms, a stoppable disobey adds 1 crack and 1 Mercy; Look cancels it with no crack; lag 2 works; `undoLock` refuses Undo once and shows IT REMEMBERS; reaching 5 cracks fires `glassBreak`; Undo rolls back cracks and Mercy.

A5 · Levels 5 and 7 · 2:40–3:00

Paste the v2 **Level plan** into the chat. For each level: design it, write the map, write the solution as key presses, check it move by move, then save the file.

- **Level 5, Kitchen** (`act: 2`, `lag: 2`, no Look yet): `undoLock` on turn 2; the text LET ME OUT; one `required` disobey with no shimmer (Look isn't unlocked yet). 1 crack.
- **Level 7, Library** (`act: 2`, `look: true`): a shimmer, then a `stoppable` disobey; later a shimmer, then a `required` disobey that is the only way to reach one plate; a short slip; the text IT HAS TO BE SOMEONE. 1–2 cracks, so Act 2 never passes 3 before Level 8.
- **Mirror check:** left and right swap, up and down don't. Solve the horizontal offset with walls, plates or boxes. Never build rooms where both bodies just walk the same column.
- Keep maps small: 7–9 tiles wide per side, 5–7 tall.

Done when: both solve on `rules-test.html` by following the written solutions, playing Level 5 then **Next level** to 7 so cracks carry.

A6 · Check-in · 3:00–3:15

Pull, play the live link with the team, and write down anything wrong with the rules.

A7 · Act 3 rules · 3:15–4:30

- **Loading Act 3:** person = Sam (`T`), reflection = Theo (`R`), `will` from the level, cracks 0 (cracks heal overnight).
- **Every turn** Sam takes the next `samRoute` step (walls block Sam too). Theo: `FOLLOW` mirrors Sam's step; `RESIST` stays still; `U` / `D` / `L` / `R` moves Theo his own way. Resist and Break cost 1 Will, add 1 crack and send `moment` kind `"shimmer"` (`who: "reflection"`). With 0 Will they act like Follow.
- **Sam's Look:** if Theo resists or breaks on a turn in `samLooks` → `fail` with reason `"seen"` . `samLooking` is true when the next turn is in `samLooks` .
- **Footprints:** Sam's next 3 positions; `glow: true` on the `frameTurn` step in Level 12.
- **Win in Levels 9–11:** when `samRoute` is finished and Theo is on `G` → won and `win` ; otherwise `fail` with reason `"route"` .
- **Level 8** (`swap: true`): when won, run `breakGlass()` and send `glassBreak` then `swap` instead of `win` . Cracks in `carry` go to 0.

Done when: in an Act 3 test room with `will: 2` and `samLooks: [3]` , Follow, Resist and Break all behave right; disobeying on turn 3 fails as "seen"; footprints show; finishing on `G` wins.

A8 · Levels 8 and 9 · 4:30–5:00

- **Level 8, Attic** (`act: 2` , `look: true` , `swap: true`): the hardest Act 2 puzzle. Both goals sit right next to the mirror line. No disobeys: Mara has stopped fighting. The text TOGETHER (right side, mirrored) a few turns before the end.
- **Level 9, Kitchen years later** (`act: 3` , `will: 2` , `controlCard: true`): Level 5's room, aged. Sam's route walks Level 5's solution. Theo needs exactly 1 Resist and 1 Break, at the spots where Mara disobeyed. One `samLooks` turn, never on a turn where Theo must disobey.

Done when: both solve; Level 8 ends with `glassBreak` then `swap` ; Level 9 can't be solved with fewer than 1 Resist and 1 Break.

A9 · Check-in · 5:00–5:15

Pull, play together, note what's wrong.

A10 · Level 12 and the endings · 5:15–6:30

Level 12, Attic years later (`act: 3`, `will: 5`, `final: true`, `frameTurn`): an `F` on each side; Sam's route steps onto the left `F` on `frameTurn`.

Every turn in Level 12, after moves and cracks, check **in this order** (first match wins):

1. Theo disobeyed on a `samLooks` turn → `fail` reason `"seen"`.
2. The 5th Will is spent (cracks reach 5) → `breakGlass()` → `status: "won"`, `ending: "take"`, events `glassBreak` then `ending`.
3. It's `frameTurn`, Theo is on the right `F` and cracks are 0–4 → `ending: "break"`.
4. `samRoute` finished with 0 Will spent → `ending: "stay"`.
5. `samRoute` finished with 1–4 Will spent → `text` NOT TONIGHT (right side, mirrored), then `fail` reason `"not-tonight"`.

- **Mercy hint:** if `carry.mercy` is 1 or more, send `text` TOGETHER on the turn before `frameTurn`.
- Every `ending` event includes `mercy`.
- **Design rule:** start the two bodies offset, so just following never brings Theo to his `F`. Reaching it on `frameTurn` must take at least 3 Will (never 5), so Stay and Break can never happen together.

Done when: all five outcomes work on `rules-test.html`: Follow every turn → Stay; the intended 3-Will path → Break; spend all 5 → Take; spend 1–2 and miss the frame → NOT TONIGHT and restart; disobey while Sam looks → seen.

A11 · Bug fixes · 6:30–7:30

Fix only what the team reports in your files. No new features.

A12 · Final push · 7:30–8:00

Push everything, then play the whole game once on the live link.

Person B — Screens & story

Your part: everything the player sees and reads. You make the starter files for the team, draw the levels from A's rules, build every screen, design the Act 1 levels and write all the story text. You only talk to A's code through `Rules.load` and `Rules.step`, and to C's code through the `Effects` functions.

B1 · Shared folder and starter files · 0:00–0:20

1. On github.com, create a repository called `mirror-world`. In **Settings → Collaborators**, add A and C.
2. Clone it in GitHub Desktop.
3. **Build the starter files**, all in one go:
   - `index.html`: a full-window page with background `#11141C`, loading every script in the contract's load order.
   - `src/levels.js`: just `const LEVELS = [];`
   - `src/levels/level01.js`: Level 1 from the contracts. `level02.js` to `level12.js`: one comment line each.
   - `src/story.js`: `STORY` with every field from the contract, placeholder text for now.
   - `src/rules.js`: a simple **stand-in** that follows the Rules interface: the mirror rule (left and right swapped, up and down the same), walls, goals, UNDO and RESTART; `carry` passed through unchanged; every other state field present with a sensible default (cracks 0, will `null`, look false, footprints empty). A replaces it.
   - `src/effects.js`: `Effects` with every function from the contract, all empty.
   - `src/game.js`: a placeholder for now.
   - `assets/images/.gitkeep` and `assets/sounds/.gitkeep`.
4. **Commit and push**, then tell A and C to pull.
5. On vercel.com: **Add New → Project** → import `mirror-world` → **Deploy**. Share the link with the team.

Done when: A and C have pulled, and the Vercel link opens with no red errors in the console (F12).

B2 · The level screen · 0:20–1:30

Build `src/game.js`:

- **Phaser setup:** a fixed 960×540 game, `pixelArt: true`, background `#11141C`, scaled to fit the window and centred.
- **Loading:** preload every image and sound from the contract. After loading, any image that's missing gets a 32×32 coloured square made in code (Theo warm `#E9B872`, reflection silver `#9FD6E0`, Sam light `#EAE4D6`, walls `#4A5570`, floor `#232A3A`). Missing sounds are skipped.

- **Level scene:** `Rules.load(level, carry)`, then draw the left half, a 16-pixel mirror strip, and the right half, centred on screen. Draw floors, walls, goals, plates, doors, spikes, frames and boxes, then the two characters: Theo left and the reflection right in Acts 1–2; Sam left and Theo right in Act 3.
- Set `scene.theo`, `scene.other` and `scene.mirror`, then call `Effects.setup(this)`, `Effects.onLevelStart(number, act)` and `Effects.setCracks(state.cracks)`.
- **Keys → inputs** as in Controls. Space sends `LOOK` only when `state.look` is true, and `FOLLOW` in Act 3. Send each input to `Rules.step`, then play the events in order, following the "B should" column of the events table. Characters slide between tiles in about 120 ms; ignore keys until all animations finish.
- **Bars:** top shows the level number and name; bottom shows key hints ("Arrows move · Z undo · R restart", plus "Space look" when Look works).
- **Undo and restart:** redraw everything straight from the new state, then `Effects.setCracks(state.cracks)`.
- **Fail:** wait 600 ms, then send `RESTART`. **Win:** `Effects.onWin()`, wait 800 ms, then load the next level with `state.carry`. Play the levels that exist in `LEVELS`, sorted by number.

Test: open `index.html` with Live Server and play Level 1 (Right 4 times).

Done when: Level 1 plays and moves on to the next level (or a "no more levels" screen); Z and R work; no red console errors.

B3 · Levels 1–3, title and story screens · 1:30–2:30

Levels: paste the v2 **Level plan** into the chat. For each level: design it, write the map and the solution as key presses, check it move by move, save it, and play it on `rules-test.html` (A's plates and boxes land around 1:45).

- **Level 1, Hallway:** keep the sample or grow it a little. A `blink` on turn 2, so there's a story moment in the first 30 seconds.
- **Level 2, Bathroom:** a plate on one side opens a door on the other. The text I'M SORRY (right side, mirrored).
- **Level 3, Bedroom:** boxes and spikes.
- **Mirror check:** left and right swap, up and down don't. Every puzzle is about the horizontal offset; never let both bodies just walk the same column.

Screens:

- **Title:** `title.png` if it exists, otherwise MIRROR WORLD in `#EAE4D6`; "Press Enter"; a small list of keys.
- **Story card:** one reusable full-screen card showing a `STORY` field, fading in, Enter to continue.
- **Game flow:** title → `intro` → `act1` → Levels 1–3 → `act2` → Levels 5–8 → swap scene → `act3` → control card → Levels 9–12 → ending → `credits` → title.

Done when: you can play from the title through Levels 1–3 with story cards in between.

B4 · The HUD and messages · 2:30–3:00

- **Act 3 strip:** "Space Follow · Shift Resist · Arrow Break" and "Will: N".
- **Footprints:** faint marks at `state.footprints`; the one with `glow: true` is warm `#E9B872` and pulses softly.
- **Sam's eye icon** over Sam when `state.samLooking` is true.
- **`text` events:** the words fade onto misty glass on that side, flipped left-to-right when `mirrored` is true, in silver `#9FD6E0`. They stay a few seconds, then fade. Call `Effects.onMoment("text", null)`.
- **`hint` events:** soft, normal text at the bottom of the screen.
- If A's Act 3 rules aren't in yet, ask Claude for a temporary test key that fills the HUD with sample values. Remove it in B9.

Done when: I'M SORRY appears mirrored on the right in Level 2; a hint shows at the bottom; the Act 3 strip, footprints and eye icon show (real or test values).

B5 · Check-in · 3:00–3:15

Pull, play the live link with the team, note what's wrong on screen.

B6 · Story text, the swap and Act 3 · 3:15–4:30

- **Write every `STORY` field** with Claude. Mysterious and bittersweet, never scary; 2–4 short lines per card. `swap` is the moment Mara wakes in Theo's bed. `controlCard` says: You can't steer anymore. Space = Follow, Shift = Resist, Arrow = Break. `credits` lists the team.
- **Swap scene (Level 8):** on `glassBreak`, call `Effects.onGlassBreak()` and `Effects.setCracks(0)`. On `swap`, pause, then the `swap` card, then the `act3` card, then Level 9.

- **Act 3 layout:** Sam on the left, Theo on the right. `scene.theo` is Theo's sprite (now on the right) and `scene.other` is Sam.
- **Level 9 control card** (`controlCard: true`): full screen, shows `STORY.controlCard`, stays until Space. Then the Space hint pulses on the HUD for the first 3 turns.

Done when: winning Level 8 runs the swap card, the Act 3 card and the control card, then Level 9 opens with Sam on the left and Theo on the right. (Before A's Levels 8 and 9 land around 5:00, test with a temporary key.)

B7 · Ending screens · 4:30–5:00

- On an `ending` event, show `STORY.endingTake`, `endingBreak` or `endingStay`. For `"break"` with `mercy` 2, also show `STORY.maraNod`. Then `credits`, then the title.
- **If there's time:** after Take Their Place, the title screen says "New Game — Sam".

Done when: each ending screen shows correctly (use a temporary test key until Level 12 exists).

B8 · Check-in · 5:00–5:15

Pull, play together, note what's wrong.

B9 · Polish · 5:15–6:30

- M mutes all sound. The title shows the controls.
- Remove every temporary test key.
- Play the whole game with A's real levels, title to ending, and fix anything on screen that looks off.
- **Only if ahead:** Level 4, the "New Game — Sam" loop, smoother transitions.

Done when: a full run from the title to an ending works with no console errors.

B10 · Bug fixes · 6:30–7:30

Fix only what the team reports in your files. No new features.

B11 · Final push and description · 7:30–8:00

Push everything. Write the game description and send it to C: a one-line pitch, how to play (keys), how the game uses "reversal", and the team's names.

Person C — Look & sound

Your part: how the game looks, sounds and feels, plus getting it submitted. You draw every sprite, write `src/effects.js` (light, glass, moments, colour, music) and test all of it on your own page, so you never wait for the real game. Keep everything soft and calm: mysterious and bittersweet, never scary.

C1 · Get set up · 0:00–0:20

- Install VS Code with **Live Server**, and **GitHub Desktop**. Accept the GitHub invite.
- Open Piskel, make a 32×32 canvas, and add the six palette colours from the contracts.
- Start drawing `theo.png`.
- Once B has pushed (about 0:10), clone `mirror-world`.
- **Done when:** you have the folder and Theo is under way.

C2 · Test page and first sprites · 0:20–1:00

Build `effects-test.html`:

- Loads Phaser 3.90 and `src/effects.js`, and preloads every image and sound from the contract (coloured squares if an image is missing).
- Draws a small room: floor and walls, Theo on the left, the reflection on the right, a 16-pixel mirror strip between. Sets `scene.theo`, `scene.other` and `scene.mirror`.
- Buttons that call each function: **Setup**, **Level start** (pick 1–12; act 1 for 1–4, 2 for 5–8, 3 for 9–12), **Cracks 0–5**, **Move Theo**, **Move reflection**, **Crack**, **Glass break**, one per **Moment** type, **Win**, and **Act 3 view** (Sam on the left, Theo on the right).

Sprites (Piskel, 32×32, export as PNG into `assets/images/`): `theo`, `reflection`, `floor`, `wall`, `goal`.

- Top-down view, soft colours from the palette, a 1-pixel dark outline.
- Theo in warm tones. The reflection is the same figure in cooler silver-blue.
- Floor and walls low-contrast, so the characters stand out. The goal is a softly glowing tile.

Done when: `effects-test.html` shows your sprites, and every button runs with no red errors (most do nothing yet).

C3 · Light and mood · 1:00–2:00

Fill in `Effects.setup(scene)`:

- Soft dark edges around the screen.
- A warm, candle-like light around `scene.theo` that follows him.
- Slow floating dust and a very faint film grain.
- A gentle silver shimmer along `scene.mirror`.
- Use Phaser 3.90's built-in effects where they help. Keep everything subtle.

Done when: after Setup, the room looks moody but calm, and the light follows Theo when you press Move Theo.

C4 · The glass and the rest of the sprites · 2:00–3:00

- **Sprites:** `sam`, `plate`, `door-closed`, `door-open`, `spikes`, `box`, `frame`. Sam has a different look from Theo, also warm.
- **`setCracks(n)`:** draw exactly `n` silver cracks on the mirror strip, each in a fixed spot (crack 1 always looks the same). 0 clears them; the same `n` twice changes nothing.
- **`onCrack(x, y)`:** a small sparkle at that spot and the crack sound; the music dips for a moment.
- **`onGlassBreak()`:** the mirror strip bursts into soft light particles, a gentle white flash, then calm.

Done when: the Cracks buttons always show exactly that many cracks, in any order; Crack and Glass break look right.

C5 · Check-in · 3:00–3:15

Pull, play the live link with the team, note what looks or sounds wrong.

C6 · Moments, colour and music · 3:15–4:30

- **`onMoment(type, body)`:** `blink` (a quick squash on the body), `wave` (a small side-to-side sway), `tilt` (a slight head tilt), `shimmer` (a wavy silver glow on the body for about a second), `slip` (a brief ripple across the screen), `text` (a soft mist puff on the glass; B draws the words). Each plays `moment.wav` softly, and the music drops out for a moment.
- **`onLevelStart(level, act)`:** fade in from black.

    - **Colour fade:** Theo is full colour in Levels 1–4, fades toward grey through Levels 5–8 while the reflection grows brighter, and stays faded in Act 3.
    - **Music:** Act 1 plays Theo's theme panned to the left; in Act 2 it drifts toward the right, level by level; Act 3 plays only Sam's theme.
- **`onMove(body)`:** a quiet step sound and a tiny dust puff.
- **`onWin()`:** a soft chime and a gentle fade.
- **Sounds:** make `step`, `crack`, `break`, `moment` and `win` in jsfxr. Keep them soft: low volume, gentle shapes, nothing sharp. Export as WAV with the contract's names.

Done when: every Moment button does something gentle; Level start 1 vs 8 shows Theo's colour fading; the music changes by act.

C7 · Music themes · 4:30–5:00

- `theme-theo.mp3` and `theme-sam.mp3`: short, gentle loops of 15–30 seconds.
- Use free music marked CC0 if the jam allows outside assets. Otherwise, ask Claude to play simple melodies in code inside `effects.js` whenever the mp3 files are missing.
- Balance volumes: music sits under the sound effects.

Done when: both themes loop with no gap, and nothing breaks if a file is missing.

C8 · Check-in · 5:00–5:15

Pull, play together, note what's wrong.

C9 · Title art and the submission page · 5:15–6:30

- **`title.png`** (960×540): the attic mirror with a figure on each side, in the soft palette, with room for the words MIRROR WORLD.
- **Submission page**, wherever the jam asks (itch.io works for web games): zip the whole `mirror-world` folder with `index.html` at the top level, upload it as an HTML game played in the browser, set the window to 960×540 with fullscreen allowed, and add B's description, the title art and tags.

Done when: a test upload plays in the browser from the page.

C10 · Bug fixes, screenshots and GIF · 6:30–7:30

- Fix only effect bugs the team reports. No new features.

- Take 3–5 screenshots (one per act, plus an ending) and a 10–20 second GIF: Act 1, a crack, the swap, Act 3.

C11 · Submit · 7:30–8:00

Pull the final version, zip it, upload, play it once in a private browser window, and submit before the deadline.

If we fall behind

Cut from the top of this list first. Claude: if a teammate is more than 20 minutes behind a step's time, suggest the next cut for their role.

Never cut: movement and the mirror rule, Level 8's swap, Act 3's Follow / Resist / Break, the crack counter, the Take Their Place and Break It Together endings. Those are the game.

Order	Cut	Whose
1	"New Game — Sam" loop, Mara's nod, Sam's notes	B
2	Music themes: use one quiet ambient loop, or none	C
3	Slip in Level 7	A
4	Mercy: no extra TOGETHER hint in Level 12	A
5	Title art: plain text title	C
6	Moments other than blink and shimmer	C
7	Level 3 and boxes (7 levels left)	B, then A
8	Sprites: coloured squares with two dots for eyes, drawn in code	C
9	Stay Behind ending (keep the NOT TONIGHT restart)	A, B
10	Level 9's Break: teach Resist only, and Break first appears in Level 12	A

Helping each other: if someone finishes early, they play the live link start to finish and send each owner a list of what's broken. Never fix someone else's file yourself.